

Explain Recursive kth Bit

Absolutely. Let's explain the **same solution again from zero**, but more precisely and in a way that makes the recursion and especially **why** $2^n - k$ completely clear.

1. First understand how $S(n)$ is created

The definition is:

```
S1 = "0"

S(n) = S(n-1) + "1" + reverse(invert(S(n-1)))
```

Let's build it.

S1

```
S1 = 0
```

S2

```
S1 = 0
```

Invert:

```
0 → 1
```

Reverse:

```
1
```

Therefore:

```
S2 = 0 + 1 + 1
    = 011
```

S3

```
S2 = 011
```

Invert:

```
011
  ↓↓↓
100
```

Reverse:

```
001
```

Therefore:

```
S3 = 011 + 1 + 001
    = 0111001
```

S4

```
S3 = 0111001
```

Invert:

```
0111001
  ↓↓↓↓↓↓
1000110
```

Reverse:

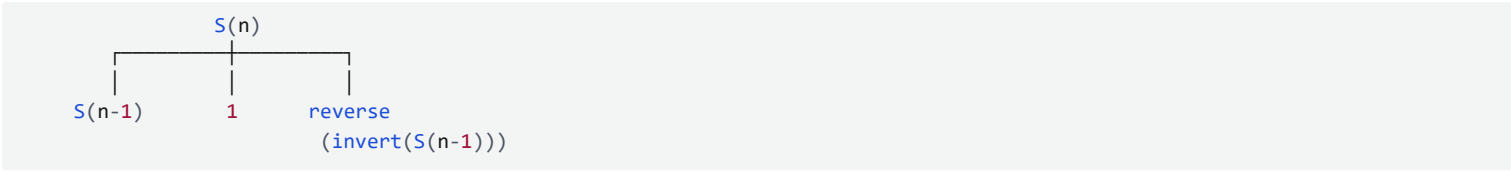
```
0110001
```

Therefore:

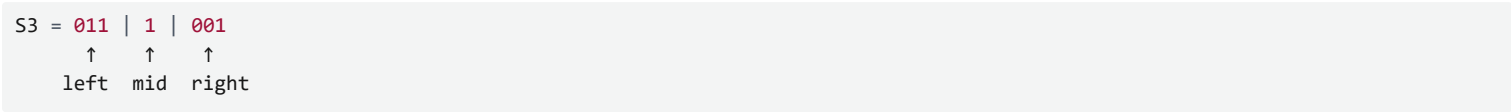
```
S4 = 0111001 + 1 + 0110001
    = 011100110110001
```

2. The important structure

Every $S(n)$ has this structure:



For example:



The length of $S(n)$ is:

```
2^n - 1
```

Therefore the middle position is:

```
(2^n - 1 + 1) / 2
= 2^(n-1)
```

So:

```
int mid = 1 << (n - 1);
```

is simply:

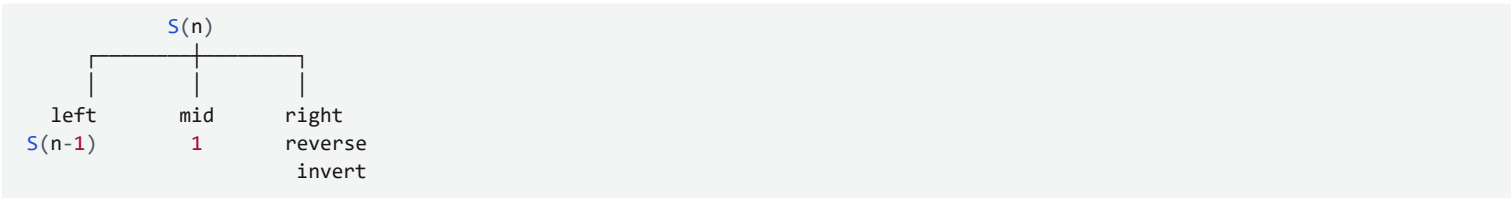
```
mid = 2^(n-1)
```

3. There are only 3 possibilities for k

Suppose we're looking for:

```
S(n)[k]
```

There are only three possibilities.



So:

Case 1: $k == mid$

We're exactly at the center.

The center is always 1.

```
if (k == mid) {
    return '1';
}
```

4. Case 2: $k < mid$

We're in the left half.

But the left half is exactly:

```
S(n-1)
```

So the problem becomes:

```
find kth bit of S(n-1)
```

The position doesn't change.

For example:

```
S4 = 0111001 | 1 | 0110001
      ↑
    left = S3
```

If we're looking for position 5:

```
S4[5]
```

position 5 is inside the left S3.

So we simply do:

```
return findKthBit(n - 1, k);
```

Notice:

```
n → n-1
k → k
```

5. Case 3: `k > mid`

This is the important case.

We're in the right half.

The right half is:

```
reverse(invert(S(n-1)))
```

So it is **not directly equal to** `S(n-1)`.

We need to find:

1. Which position of `S(n-1)` produced this position?
2. Then invert that bit.

6. Understand the right half with S3

Take:

```
S3 = 0111001
```

Positions:

```
position:  1 2 3 | 4 | 5 6 7
value:      0 1 1 | 1 | 0 0 1
```

The right half is:

```
0 0 1
```

But this came from S2:

```
S2 = 0 1 1
      1 2 3
```

Because we **reverse** it:

```
S2:      0 1 1
          ↑ ↑ ↑
          1 2 3

reverse:  1 1 0
          ↑ ↑ ↑
          3 2 1
```

Then invert:

```
1 1 0
↓ ↓ ↓
0 0 1
```

Therefore:

```

S3:

position:  1 2 3 | 4 | 5 6 7
value:     0 1 1 | 1 | 0 0 1
              ↓ ↓ ↓
S2 index:      3 2 1

```

This is the key relationship.

7. Why is the new position $2^n - k$?

Let's take:

```
n = 3
```

The total length is:

```
2^3 - 1 = 7
```

The right half positions are:

```
5 6 7
```

And they correspond to `s2` positions:

```
3 2 1
```

So:

```

S3 position 5 → S2 position 3
S3 position 6 → S2 position 2
S3 position 7 → S2 position 1

```

Now notice:

```

8 - 5 = 3
8 - 6 = 2
8 - 7 = 1

```

And:

```
8 = 2^3
```

Therefore:

```
new_k = 2^n - k
```

In C++:

```
(1 << n) - k
```

because:

```
1 << n = 2^n
```

8. Why not $2^n - 1 - k$?

This is a common confusion.

Suppose:

```
n = 3
```

The string has positions:

```
1 2 3 4 5 6 7
```

For the right half:

```

5 → 3
6 → 2
7 → 1

```

We need:

$$5 \rightarrow 3$$

So:

$$8 - 5 = 3$$

not:

$$7 - 5 = 2$$

That's why we use:

$$2^n - k$$

The +1 comes from converting the reversed 1-based positions.

9. Why do we invert the answer?

The right half isn't just reversed.

It is:

```
reverse(invert(S(n-1)))
```

Therefore, after finding the corresponding bit in `S(n-1)`, we must invert it.

Suppose recursion gives:

```
ans = '0'
```

Then the corresponding right-half bit is:

```
'1'
```

And if:

```
ans = '1'
```

then:

```
'0'
```

That's why:

```
return ans == '0' ? '1' : '0';
```

10. Now understand the complete code

```
class Solution {
public:
    char findKthBit(int n, int k) {

        if (n == 1) {
            return '0';
        }

        int mid = 1 << (n - 1);

        if (k == mid) {
            return '1';
        }

        if (k < mid) {
            return findKthBit(n - 1, k);
        }

        char ans = findKthBit(n - 1, (1 << n) - k);

        return ans == '0' ? '1' : '0';
    }
};
```

Let's understand every line.

Line 1

```
if (n == 1) {  
    return '0';  
}
```

We eventually reach:

```
S1 = 0
```

So if:

```
n = 1
```

the answer is directly:

```
0
```

This is our **base case**.

Line 2

```
int mid = 1 << (n - 1);
```

This calculates:

```
2^(n-1)
```

which is the middle position.

For example:

```
n = 4  
  
mid = 1 << 3  
     = 8
```

Because:

```
S4 has length 15
```

and position:

```
8
```

is the middle.

Line 3

```
if (k == mid) {  
    return '1';  
}
```

The middle bit is always 1.

Done.

No recursion required.

Line 4

```
if (k < mid) {  
    return findKthBit(n - 1, k);  
}
```

We're in the left half.

Left half = $S_{(n-1)}$.

So:

```
n → n-1  
k → k
```

Lines 5-6

```
char ans = findKthBit(n - 1, (1 << n) - k);

return ans == '0' ? '1' : '0';
```

We're in the right half.

Therefore:

```
n → n-1
```

and because the right half is reversed:

```
k → 2^n - k
```

Then because it is inverted:

```
0 → 1
1 → 0
```

11. Complete dry run: n = 4, k = 11

We know:

```
S4 = 011100110110001
```

We want:

```
S4[11]
```

which is:

```
1
```

Let's see how recursion finds it.

Call 1

```
findKthBit(4, 11)
```

Calculate:

```
mid = 1 << 3
     = 8
```

We have:

```
k = 11
mid = 8
```

Therefore:

```
11 > 8
```

We're in the right half.

Convert k:

```
2^4 - 11
= 16 - 11
= 5
```

So:

```
findKthBit(4, 11)
    ↓
findKthBit(3, 5)
```

And because we went through the inverted right half, we'll invert the result later.

12. Call 2

```
findKthBit(3, 5)
```

Calculate:

```
mid = 1 << 2
     = 4
```

We have:

```
k = 5
mid = 4
```

Therefore:

```
5 > 4
```

Again right half.

Convert:

```
2^3 - 5
= 8 - 5
= 3
```

So:

```
findKthBit(3,5)
  ↓
findKthBit(2,3)
```

Again, we'll invert while returning.

13. Call 3

```
findKthBit(2,3)
```

Calculate:

```
mid = 1 << 1
     = 2
```

We have:

```
k = 3
mid = 2
```

Again:

```
3 > 2
```

Right half.

Convert:

```
2^2 - 3
= 4 - 3
= 1
```

So:

```
findKthBit(2,3)
  ↓
findKthBit(1,1)
```

14. Call 4 – Base case

```
findKthBit(1,1)
```

Since:

```
n == 1
```

we return:

```
'0'
```

So:

```
find(1,1)
  ↓
  '0'
```

Now the recursion starts **returning backward**.

15. Returning to find(2,3)

We originally had:

```
find(2,3)
```

and $k = 3$ was in the right half.

We got:

```
ans = '0'
```

But right half is inverted.

Therefore:

```
0 → 1
```

So:

```
find(2,3) = '1'
```

16. Returning to find(3,5)

We had:

```
find(3,5)
```

Again, $k = 5$ was in the right half.

The recursive result is:

```
'1'
```

Invert it:

```
1 → 0
```

Therefore:

```
find(3,5) = '0'
```

17. Returning to find(4,11)

Again, $k = 11$ was in the right half.

We received:

```
'0'
```

Invert:

```
0 → 1
```

Therefore:

```
find(4,11) = '1'
```

Final answer:

```
'1'
```

18. The entire recursion in one view

```
find(4,11)
|
| k > 8
| new k = 16 - 11 = 5
```

```

↓
find(3,5)
  |
  | k > 4
  | new k = 8 - 5 = 3
  ↓
find(2,3)
  |
  | k > 2
  | new k = 4 - 3 = 1
  ↓
find(1,1)
  ↓
  '0'

```

Now return:

```

find(1,1)
  ↓
  '0'

find(2,3)
  ↓ invert
  '1'

find(3,5)
  ↓ invert
  '0'

find(4,11)
  ↓ invert
  '1'

```

Final:

```
'1'
```

19. The main thing you should remember

Don't try to memorize the entire code.

Remember the **structure**:

```
S(n) = S(n-1) + 1 + reverse(invert(S(n-1)))
```

Therefore:

If k is in the left:

```
S(n-1)
```

So:

```
findKthBit(n - 1, k)
```

If k is in the middle:

```
1
```

So:

```
return '1';
```

If k is in the right:

The right side is:

```
reverse(invert(S(n-1)))
```

Therefore:

1. Reverse position
2. Go to S(n-1)
3. Invert answer

which becomes:

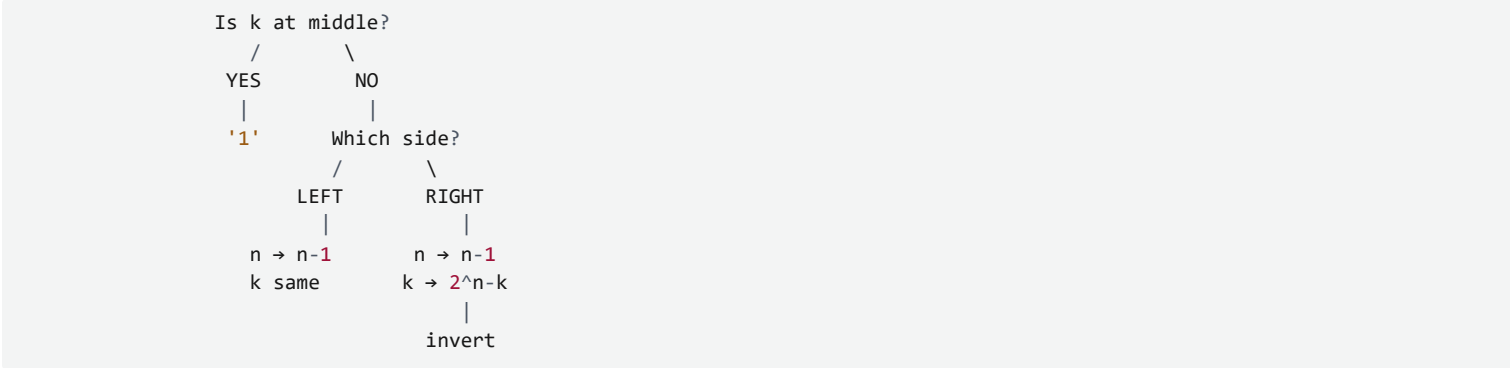
```
char ans = findKthBit(n - 1, (1 << n) - k);
return ans == '0' ? '1' : '0';
```

20. Final mental model

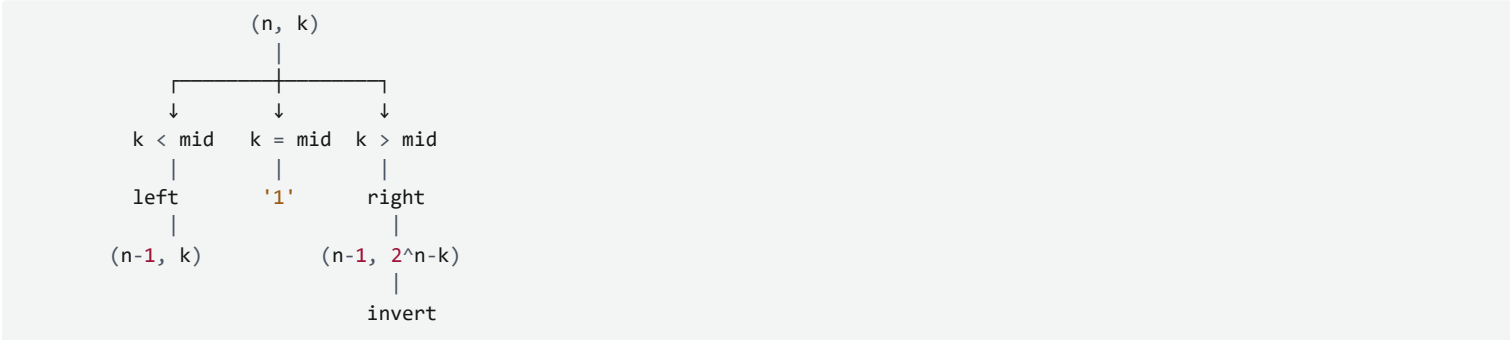
Whenever you see:

```
find(n, k)
```

ask:



So the entire algorithm is basically:



That's the whole trick.

The algorithm never constructs the huge string. It only figures out **where the required position came from** in the smaller string and keeps doing that until reaching `s1 = "0"`.